NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC1007: Data Structures & Algorithm

Linked List



Presented by Dr Owen Noel Newton Fernando

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Good morning everyone, and welcome to today's session on linked lists. In this lecture, we'll explore one of the most important dynamic data structures in computer science and see how it overcomes some of the limitations of arrays.

We'll begin by understanding how linked lists organise data using connected nodes before learning how to implement them in Python.

As we progress, we'll work through the fundamental operations, including traversal, searching, insertion, and removal, and by the end of the session, you'll have a solid understanding of how linked lists work and why they are widely used in real-world software development.



Objectives

- Introduce linked-list concepts and explain their purpose.
- Compare the structure and behaviour of linked lists and arrays.
- Explain nodes, references, and the organisation of singly linked lists.
- Present the Python implementation of ListNode and LinkedList classes.
- Demonstrate traversal, search, insertion, removal, and size management.
- Introduce task scheduling as a practical application of linked lists.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Before we get into the technical details, let's take a moment to see where today's lesson is heading. Having a clear roadmap will help you understand how each concept connects as we move through the lecture.

We'll begin by exploring the fundamental ideas behind linked lists and comparing them with arrays. After that, we'll look at how nodes and references are organised before building the ListNode and LinkedList classes together in Python.

Once the basic structure is in place, we'll move on to the essential operations you'll use most often. By the end of today's session, you'll understand how to traverse, search, insert, remove, and manage linked lists, and you'll also see how these ideas are applied in practical systems such as task scheduling.

Learning Outcome

Upon successful completion of this topic, students will be able to:

- Explain how nodes and references form a singly linked list.
- Compare linked lists with arrays and identify appropriate uses for each.
- Trace linked-list traversal and predict the resulting sequence.
- Develop ListNode and LinkedList classes in Python.
- Implement display, search, insertion, and removal operations.
- Handle empty lists, invalid indices, and boundary positions correctly.
- Maintain and retrieve the number of nodes using a size attribute.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Now that we've looked at today's objectives, let's focus on what you should be able to achieve by the end of this lesson. These learning outcomes describe the knowledge and practical skills you'll develop as we work through the topic together. Rather than simply understanding the theory, you'll learn how linked lists are organised and when they are a better choice than arrays.

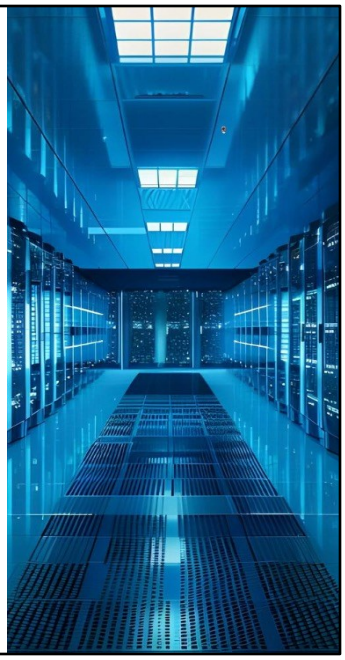
As we move through the lecture, you'll learn how nodes and references work together to build a singly linked list. We'll also trace linked-list traversal step by step before developing the ListNode and LinkedList classes in Python, giving you a clear understanding of how the concepts are translated into working code.

By the end of today's session, you'll be able to implement the core linked list operations with confidence. You'll know how to display, search, insert, and remove nodes, handle important boundary cases such as empty lists and invalid indices, and maintain the size of the linked list efficiently using a dedicated size attribute.

Contents

- Linked-List Fundamentals and Comparison with Arrays
- Linked-List Types and Singly Linked Lists
- Python Implementation Using ListNode and LinkedList
- Traversal, Display, and Node Search
- Node Insertion and Removal
- Maintaining the Linked-List Size
- Refining the LinkedList Class
- Real-World Application: Task Scheduling
- Summary

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



Before we start exploring linked lists, it's worth taking a quick look at today's roadmap. The topics are arranged in a logical order, so each section builds on the ideas introduced in the previous one.

We'll begin by understanding the fundamentals of linked lists and comparing them with arrays to see when each structure is most appropriate. After that, we'll focus on singly linked lists and learn how to represent them in Python using the ListNode and LinkedList classes.

As the lecture progresses, we'll implement the key linked list operations that you'll use most often. We'll finish by refining our implementation, looking at a real-world task scheduling example, and bringing everything together in a short summary of the main concepts.

Linked List Fundamentals and Comparison with Arrays

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Every data structure is designed to solve a particular problem, and linked lists are no exception. Before we start writing any code, it's important to understand why linked lists were introduced and how they differ from arrays, which you're already familiar with.

In this section, we'll explore the fundamental ideas behind linked lists and the way they organise data in memory. We'll compare their characteristics with arrays and discuss the strengths and limitations of each approach rather than assuming one is always better than the other.

As we work through these concepts, try to think about the situations where flexibility is more important than fast random access. That perspective will help you understand why linked lists remain an essential data structure and prepare you for the implementation and operations we'll cover next.

Why Linked Lists?

- Arrays store elements in contiguous memory locations.
- Array elements can be accessed directly using an index.
- Inserting or deleting elements may require shifting other elements.
- Linked lists store elements as nodes connected by references.
- Nodes can be added or removed without reorganising the entire structure.
- Linked lists support dynamic growth but require sequential access.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's begin with a simple question: why do we need another data structure when arrays already work so well? Arrays are excellent for storing data and providing fast access using an index, but they become less efficient when elements need to be inserted or removed frequently.

The main reason is that arrays store their elements in contiguous memory locations. When an element is inserted or deleted, many other elements may need to be shifted, whereas linked lists avoid this by storing data in separate nodes connected through references.

This difference gives linked lists much greater flexibility when the amount of data changes over time. The trade-off is that we lose direct access to elements, so we must move through the list one node at a time, making it important to understand both the advantages and limitations of this approach.

Arrays vs Linked Lists

Feature	Array	Linked List
Memory layout	Contiguous locations	Non-contiguous nodes
Size	Fixed or resized by copying	Grows and shrinks dynamically
Element access	Direct access using an index	Sequential traversal
Memory overhead	Low	Additional reference per node
Cache performance	Generally better	Generally lower
Implementation	Relatively simple	Requires reference management

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's compare these two data structures side by side to understand where each one performs best. Rather than thinking of one as better than the other, it's more useful to recognise that each has its own strengths and is designed for different situations.

The biggest difference lies in how data is stored and accessed. Arrays keep elements in contiguous memory, allowing direct access through an index, while linked lists store nodes in separate memory locations and reach each element by following references one node at a time.

These design choices influence everything from memory usage to overall performance. Arrays are generally simpler to implement and provide better cache performance, whereas linked lists offer greater flexibility for frequent insertions and deletions, making the right choice dependent on the requirements of the application rather than the data structure itself.

Linked List: Core Operations

- **Traverse/Display**: Start at the head, process each node, and follow references until None.
- **Search**: Traverse sequentially to locate a node by value, key, or position.
- **Insert**: Create a node and update references to place it at the front, middle, or back.
- **Delete/Remove**: Locate the node, reconnect surrounding references, and update the head or tail when needed.
- **Size**: Count nodes through traversal or maintain a size attribute after insertions and deletions.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

At this stage, let's focus on the core operations that make linked lists useful in practice. Although the underlying structure is quite simple, these operations allow us to store, access, and modify data efficiently in a wide range of applications.

We'll start by learning how to traverse the list, which simply means visiting one node after another until we reach the end. Building on that, we'll see how searching follows the same path, while insertion and removal work by updating the links between nodes rather than moving the data itself.

Another important operation is determining the size of the linked list. We can either count the nodes by traversing the list whenever needed or maintain a dedicated size attribute that is updated after every insertion and deletion, making the operation much more efficient.

Types of Linked Lists

Type	Node References	End Condition	Traversal
Singly Linked List	Reference to the next node	Final node points to None	Forward only
Doubly Linked List	References to the previous and next nodes	First and final outer references point to None	Forward and backward
Circular Singly Linked List	Next node	Tail's next reference points to the head	Forward in a continuous cycle
Circular Doubly Linked List	Previous and next nodes	Tail's next points to the head; head's previous points to the tail	Forward and backward in a continuous cycle

Singly linked lists prioritise simplicity, doubly linked lists support bidirectional movement, and circular linked lists support continuous traversal.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Not all linked lists are organised in the same way, and each variation is designed for different requirements. The main difference lies in how the nodes are connected and how we are allowed to move through the list during traversal.

The simplest version is the singly linked list, where each node stores only a reference to the next node, allowing movement in one direction. A doubly linked list adds a reference to the previous node, making it possible to move forwards and backwards, although this comes at the cost of extra memory and slightly more complex updates.

Circular linked lists take a different approach by connecting the final node back to the first node instead of ending with None. As a result, traversal can continue in a continuous loop, making circular linked lists particularly useful for applications such as task scheduling, round-robin processing, and buffering, where data needs to be processed repeatedly.

Singly Linked Lists and Core Operations

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

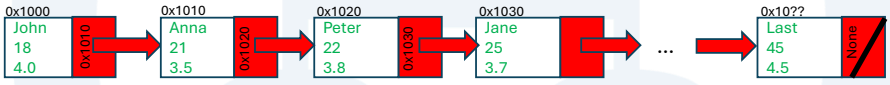
Having explored the different types of linked lists, we're now ready to focus on the one you'll encounter most often. The singly linked list is the simplest version, making it the ideal place to learn how linked lists work before moving on to more advanced variations.

Each node in a singly linked list stores some data together with a reference to the next node in the sequence. Because every connection moves in only one direction, we always begin at the first node and follow each reference until we reach the end of the list.

This structure provides everything we need to understand the fundamental linked list operations. Over the next few slides, we'll see how to create nodes, traverse the list, search for data, insert and remove nodes, and maintain the size of the linked list using efficient Python implementations.

Singly Linked Lists

- A singly linked list consists of nodes arranged in a linear sequence.
- Each node stores:
  - **Data**: the value or object associated with the node.
  - **Next**: a reference to the following node.



- Nodes may occupy non-contiguous memory locations.
- References preserve the logical order of the nodes.
- Traversal proceeds in one direction, from one node to the next.
- The final node's next reference is None, marking the end of the list.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's use this diagram to see how a singly linked list is organised in memory. Although the nodes appear as a single sequence, they do not have to be stored next to one another. Instead, each node contains the actual data together with a reference that points to the next node in the list.

Notice that the arrows, rather than the memory addresses, determine the order of the nodes. Starting from John, we follow the references to Anna, Peter, Jane, and finally the last node, even though the nodes may be located in completely different memory locations.

One important characteristic of a singly linked list is that traversal always moves in one direction. We continue following the next reference until we reach the final node, whose next reference is None, indicating that there are no more nodes and that we have reached the end of the linked list.

Implementation of a linked list in Python

- The ListNode class stores data and a reference to the next node.

```
class ListNode:
   def __init__(self, data):
        self.data = data
        self.next = None
```

- `def __init__(self, data):` Constructor method that initialises a new ListNode object.
  - `self` refers to the current node instance being created.
  - `data` is the parameter that stores the node's value.
  - `self.next`: is an instance variable that stores a reference to the next node. It is initially set to None and is updated later when nodes are linked together.
- Node Components:
  - `self.data = data` → Stores the value in the node.
  - `self.next = None` → Creates a reference to the next node. It is initially **None** and is updated when the node is linked to another node.

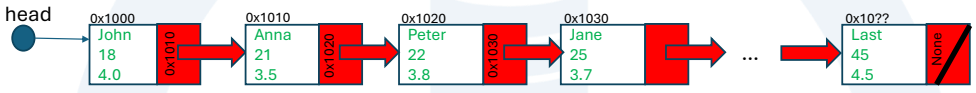
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's move from the concept to the actual Python implementation of a node. Every linked list is built from individual nodes, so the first step is to define a class that represents a single node and the information it needs to store.

The ListNode class contains a constructor, __init__, which is called whenever a new node is created. Here, self refers to the current node instance, while data stores the value held by the node. The constructor also **creates an instance variable named self.next** and initialises it to None. We define this variable ourselves because every node needs a place to store a reference to the next node, even though the node is not connected to another node when it is first created.

Now focus on the two assignment statements because they define the structure of every node in the linked list. The statement self.data = data stores the node's value, while self.next = None creates a placeholder for the reference to the next node. Later, when we insert nodes into the linked list, we replace None with a reference to another ListNode, allowing the nodes to be connected together to form a linked list.

Implementation of a linked list in Python



- A linked list class is used to create and manage a list of nodes.
- The **head reference** is essential to locate the first node in the list.
- An additional **reference**, such as a tail pointer, can improve efficiency by pointing to the last node.
- The class supports operations such as insertion, deletion, and traversal to manage the list effectively.

```
class LinkedList:
    def __init__(self):
        self.head = None
```

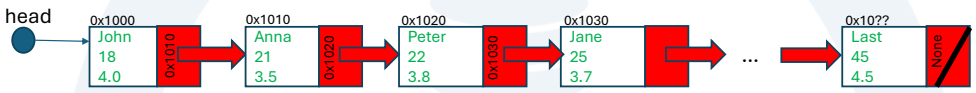
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Creating individual nodes is only the first step; we also need a way to manage them as a complete linked list. That's the role of the LinkedList class, which keeps track of the nodes and provides the operations needed to work with the entire structure.

The most important member of this class is the head reference because it always points to the first node in the list. When a new linked list is created, self.head is set to None, indicating that the list is empty, and once the first node is added, the head is updated to point to that node.

Some implementations also maintain a tail reference to locate the last node more efficiently, although this is optional. With these references in place, the LinkedList class can support operations such as traversal, searching, insertion, and deletion while ensuring that all the connections between nodes remain correct.

Common Operations of a Simple Singly Linked List



- **Display** each element
- **Search** for a node
- **Add** a new node
- **Remove** a node
- Determine the **size** of the linked list

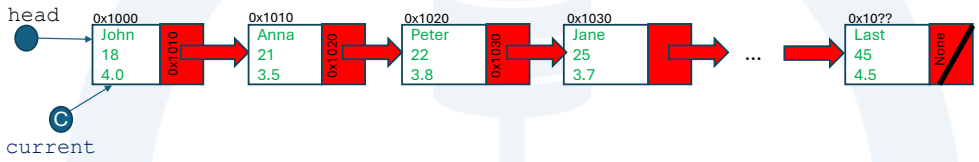
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Now that we've built the basic linked list structure, it's time to explore the operations we'll perform on it. These operations form the foundation of almost every linked list application, and you'll see the same ideas repeated in many other data structures as well.

Most of these operations begin at the head of the list and follow the references from one node to the next. We'll learn how to display every element, search for a particular node, add new nodes, and remove existing ones without breaking the connections between the remaining nodes.

We'll also look at how to determine the number of nodes in the linked list. This can be done by counting the nodes during traversal or, as we'll see later, by maintaining a size attribute that is updated whenever the list changes, making the operation much more efficient.

Display each element in the linked list



Display each element in the linked list

```
def display(self):
    current = self.head
    while current:
        print(current.data, end=" -> ")
        current = current.next
    print("None")
```

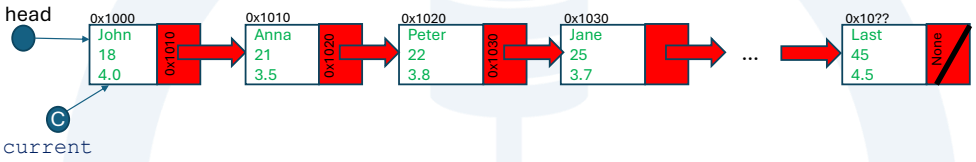
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's begin with one of the simplest yet most important linked list operations: displaying the elements. This operation introduces the basic idea of traversal, where we visit every node in the list one at a time, making it the foundation for many other linked list operations.

To perform the traversal, we create a temporary reference called current and initialise it with self.head, which points to the first node. The while loop continues as long as current refers to a valid node, printing the node's data before moving to the next node using current = current.next.

Notice that the original head reference never changes during this process. Instead, the temporary current reference moves from one node to the next until it becomes None, signalling that the end of the linked list has been reached and the traversal is complete.

Search the node at index i



- Looking for the ith node in the list
  - Given the **head reference** of the linked list and **index i**
  - Return the pointer to the **ith** node
  - **None** will be return if index **i** is out of the range or the linked list is empty

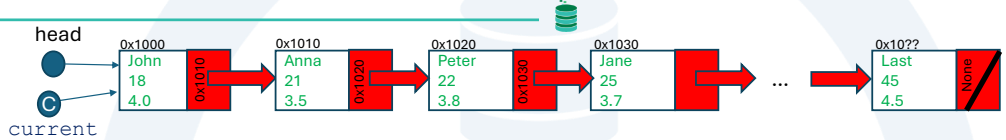
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

The next operation we'll explore is searching for a node at a specific index. Unlike an array, a linked list cannot jump directly to the required position because its nodes are connected by references rather than stored in consecutive memory locations.

To find a particular node, we always begin at the head reference and move through the list one node at a time. As we traverse the list, we count each step, and when the current position matches the required index, we return a reference to that node.

Of course, we also need to consider situations where the search cannot be completed successfully. If the list is empty, the index is negative, or the requested position lies beyond the end of the list, there is no valid node to return, so the method simply returns None.

Search the node at index i




```
1  def findNode(self, index):
2      if self.head is None or index < 0:
3          return None
4
5      current = self.head
6      while index > 0:
7          current = current.next
8
9          if current is None:
10             return None
11         index -= 1
12     return current
```

- Start from **head** (the reference to the first node).
- If the list is **empty** → **return None**.
- Traverse node by node, **decrementing index**.
- If you reach the end (None) **before finishing** → **return None.**
- When **index reaches 0** → return the reference to the current node.

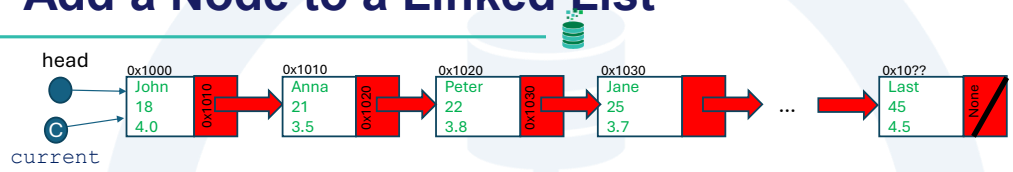
© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's walk through the findNode method to understand how it locates a node at a given index. The method first checks whether the linked list is empty or whether the supplied index is negative. If either condition is true, there is no valid node to return, so the method immediately returns None.

For a valid search, the temporary reference current is initialised to self.head, which represents the first node in the list. The while loop then moves current to the next node while decreasing the index by one after each step, gradually bringing the index closer to zero.

Pay particular attention to the condition inside the loop because it prevents the traversal from moving beyond the end of the list. If current becomes None before the index reaches zero, the requested position does not exist, so the method returns None; otherwise, when the index reaches zero, current points to the required node and that reference is returned.

Add a Node to a Linked List



Insert a Node into a Linked List

Given:

- A pointer to the head of the linked list
- The **index (i)** where the new node should be inserted
- The item (data) to be stored in the new node

Return:

- **SUCCESS (1)** if the insertion is successful
- **FAILURE (0)** otherwise

Algorithm Overview

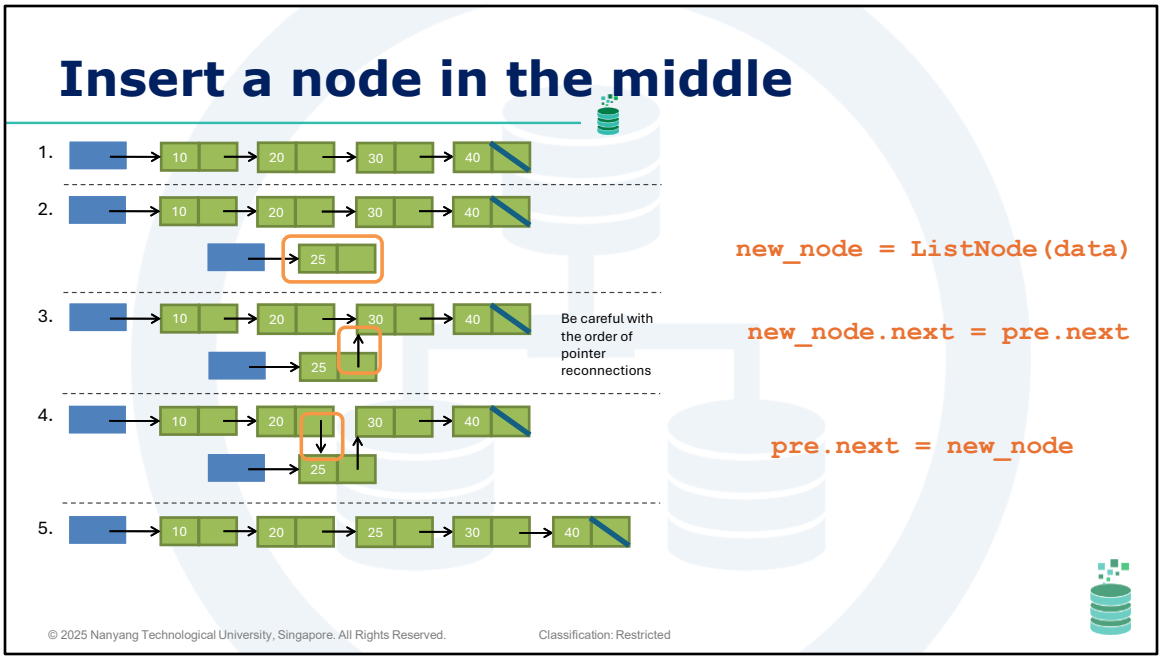
1. Create a new node containing the given item.
2. Insert the new node at the specified position:
   - **Front** *(index = 0)*
   - **Middle** *(0 < index < size)*
   - **Back** *(index = size)*

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Insertion is one of the biggest advantages of using a linked list over an array. Instead of shifting many elements to create space, we simply create a new node and connect it to the existing list by updating the appropriate references.

To perform the insertion, we need three pieces of information: the head reference, the index where the new node should be placed, and the data it will store. Depending on the specified index, the node can be inserted at the front of the list, in the middle, or at the back when the index matches the current size of the linked list.

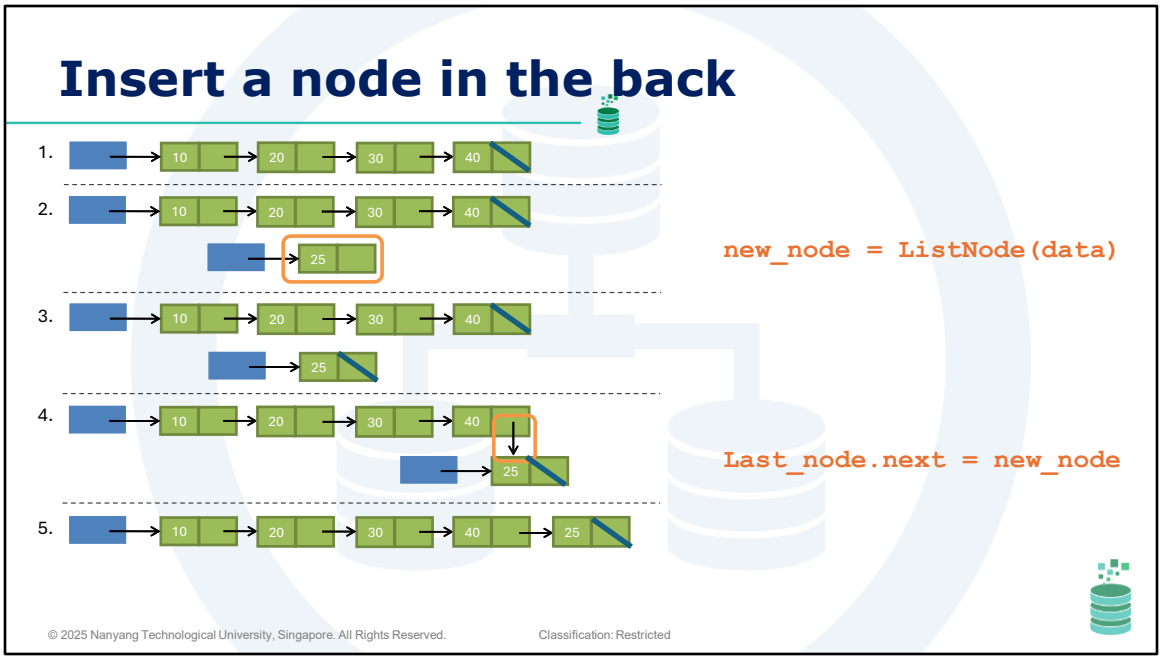
Carefully updating the references is the key to a successful insertion. If the links are modified correctly, the new node becomes part of the list without affecting the remaining nodes, and the method returns SUCCESS (1); otherwise, if the position is invalid, it returns FAILURE (0).



Middle insertion is slightly more involved because we need to reconnect the existing nodes without breaking the linked list. The order of these pointer updates is extremely important, as changing the wrong reference first can disconnect part of the list and make some nodes inaccessible.

The process begins by creating the new node and setting new_node.next = pre.next. At this point, the new node points to the node containing 30, while the node containing 20 still points to 30, so the original list remains unchanged and no connections are lost.

Only after that do we update pre.next = new_node to complete the insertion. The node containing 20 now points to the new node containing 25, which already points to 30, producing the final sequence 10 → 20 → 25 → 30 → 40 while preserving every existing node in the correct order.



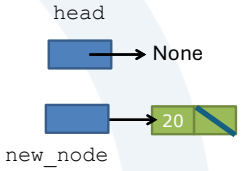
Appending a node to the end of the linked list follows the same basic idea, but this time we insert the new node after the current last node. We begin by creating the new node using new_node = ListNode(data), and its next reference is initially set to None because it will become the final node in the list.

The next step is to traverse the linked list until we reach the last node, whose next reference is None. Once that node is found, we update its reference using the statement last_node.next = new_node, connecting the new node to the end of the existing list.

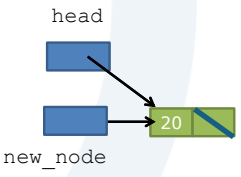
Notice that only a single reference needs to be updated to complete the insertion. After this assignment, the new node becomes the last node in the linked list, and because its next reference remains None, it correctly marks the end of the sequence while preserving all the existing connections.

Case: Empty Linked List

- Boundary Case: Empty Linked List
  - self.head is None.
  - The new node becomes the first node in the list.
  - Only the head reference needs to be updated.




```
# Step 1: Create a new node
 new_node = ListNode(data)
# Step 2: Point new node to the current head
 new_node.next = self.head
# Step 3: Update the head to point to the new node
 self.head = new_node
```



- The head reference changes because the first node of the linked list changes.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's begin with an important boundary case before looking at the general insertion algorithm. An empty linked list has no nodes, so self.head does not point to any node and simply contains the value None.

Interestingly, we do not need a special insertion algorithm for this situation. We create the new node first and execute new_node.next = self.head, which means the new node's next reference is simply set to None because there is no existing first node.

Finally, we update the head by executing self.head = new_node. The head now points to the newly created node, making it the first node in the list, and these same three steps also work when the linked list already contains one or more nodes.

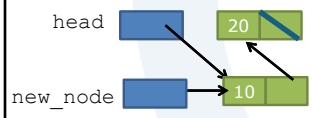
Case: Inserting at the Front (Index 0)

- Before insertion



The head reference must be updated because the first node in the linked list changes.

- After insertion




```
# Step 1: Create a new node
new_node = ListNode(data)
# Step 2: Point new node to the current head
new_node.next = self.head
# Step 3: Update the head to point to the new node
self.head = new_node
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Let's now see what happens when we insert a node at the front of a linked list. In this example, the current list starts with the node containing 20, and we want to insert a new node containing 10 at index 0, so the head reference must change.

The first step is to create the new node and connect it to the current first node. We do this with the statement new_node.next = self.head, which preserves the existing linked list by making the new node point to the node containing 20 before any references are updated.

Finally, we update the head using self.head = new_node. The head now points to the new node containing 10, producing the sequence 10 → 20 → None, while the remaining nodes and their connections stay exactly the same.

Insert a node at the front

- Why can the same algorithm handle both cases?
  - **Case 1: Empty Linked List**

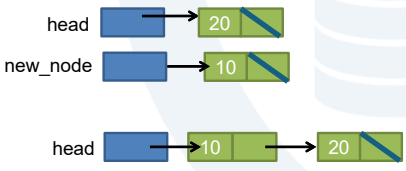



```
# Step 1
new_node = ListNode(data)

# Step 2
new_node.next = self.head

# Step 3
self.head = new_node
```

  - **Case 2: Insert at Index 0**



Only the head reference changes; the insertion algorithm remains the same.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

Understanding why this algorithm works in both situations makes the implementation much easier to remember. Whether the linked list is empty or we are inserting at index 0, the same three statements perform the insertion correctly.

For an empty linked list, self.head is None, so the statement new_node.next = self.head simply makes the new node point to None. Updating self.head = new_node then makes the new node the first and only node in the linked list.

The same sequence of statements also works when the linked list already contains nodes. In this case, new_node.next = self.head preserves the existing first node before self.head is updated, allowing the new node to become the first node while keeping the remainder of the linked list connected. Only the head reference changes, while the insertion algorithm itself remains exactly the same.

Insert node at given index

```
def insertNode(self, index, data):

    # Check for an invalid index
    if index < 0:
        return False

# Create a new node with the given data
    new_node = ListNode(data)

    # Insert at the front
    if index == 0:
        new_node.next = self.head
        self.head = new_node
        return True

    # Start from the head
    current = self.head
    position = 0

    # Move to the node before the insertion position
    while current is not None and position < index - 1:
        current = current.next
        position += 1

    # Index is beyond the end of the list
    if current is None:
        return False

    # Insert the new node:
    new_node.next = current.next
    current.next = new_node

    return True
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

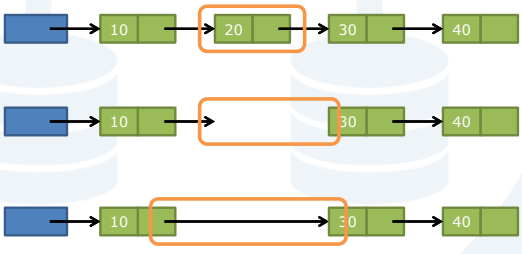
This implementation brings together all the insertion cases into a single method. Rather than writing separate functions for front, middle, and back insertion, one algorithm can handle every valid insertion position in the linked list.

The method first creates the new node and then checks whether the insertion is at the front of the list. If the index is zero, the new node becomes the new head. Otherwise, the algorithm starts from the head and traverses the linked list until it reaches the node immediately before the required insertion position.

Once the predecessor has been found, the references are updated in two simple steps. The new node first points to the successor, and the predecessor is then updated to point to the new node. If no valid predecessor is found, the method returns False; otherwise, the insertion is completed successfully and the method returns True.

Remove a Node

- To remove a node from a linked list, we must update the reference in its predecessor.
- In a singly linked list, references can only be followed in the forward direction.
- Removing the first node is a special case because it has no predecessor. The head reference must be updated instead.
- We will consider three removal cases:
  1. Front
  2. Middle
  3. Back



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Now that we've covered insertion, let's see how node removal works in a linked list. Unlike insertion, we are not creating a new reference; instead, we must update an existing reference so the remaining nodes stay connected after the target node is removed.

In a singly linked list, we cannot move backwards because each node stores only a reference to the next node. For that reason, we must start at the head and traverse the list until we reach the predecessor of the node we want to remove.

One case needs special attention, and that is removing the first node. Since the head node has no predecessor, we simply update the head reference, then we'll look at removing nodes from the front, middle, and back one case at a time.

Remove a Node

```
def removeNode(self, index):
    # Check for an invalid index
    if self.head is None or index < 0:
        return False

    # Remove the first node
    if index == 0:
        current = self.head
        self.head = current.next
        del current
        return True

    # Start from the head
    current = self.head
    position = 0

    # Move to the node before the removal position
    while current is not None and position < index - 1:
        current = current.next
        position += 1

    # Index is beyond the end of the list
    if current is None or current.next is None:
        return False

    # Remove the node by updating the references
    target = current.next
    current.next = target.next

    del target

    return True
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

This method follows the same overall structure as insertion, but this time our goal is to remove an existing node. It begins by checking whether the linked list is empty or the supplied index is invalid, because in either case there is no valid node to remove and the method simply returns False.

When the index is zero, the first node is handled as a special case. The current head is stored temporarily, the head is updated to point to the second node, the original first node is deleted, and the method returns True.

For all other positions, the algorithm starts from the head and moves to the node immediately before the removal position. If the index is beyond the end of the list, the method returns False; otherwise, target is assigned to current.next, current.next is updated to target.next, the target node is deleted, and the method returns True.

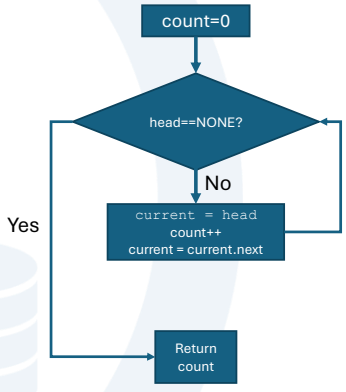
Size of The LinkedList (not smart solution)

Given

- the head reference of the linked list

Return the number of nodes in the linked list

1. Declare a counter and initialize it to zero
2. Check the reference whether is None or not
3. Increase the counter
4. Current move to next node
5. Repeat step 2
6. Return the counter



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

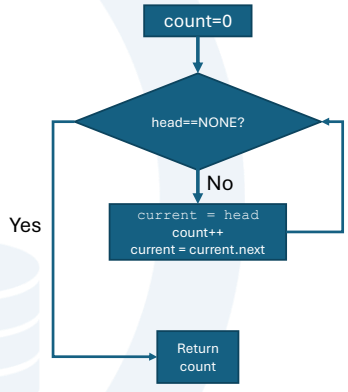
This example shows the most straightforward way to determine the size of a linked list. The idea is simple: we visit every node one by one and count how many nodes we encounter before reaching the end of the list.

First, we initialise a counter to zero and set a temporary reference, called current, to the head of the linked list. While current is not None, we increase the counter by one and move current to the next node, repeating this process until there are no more nodes to visit.

This approach always produces the correct size because every node is counted exactly once. However, each time we need the size, we must traverse the entire linked list, giving a time complexity of O(n). Although this works well, it becomes less efficient when the list is large or the size is requested frequently, which leads us to a smarter solution in the next slide.

Size of The LinkedList (not smart solution)

```
1  def getSize(self):
2
3      # Initialize counter
4      count = 0
5
6      # Start at head
7      current = self.head
8
9      # Traverse the list
10     while current is not None:
11         count += 1  # Increase counter
12         current = current.next  # Move to next node
13
14     return count  # Return final count
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This example walks us through the getSize method to see how it counts the number of nodes in a linked list. Line 1 defines the method, while line 4 creates the variable count and initialises it to zero.

Next, line 7 creates the temporary reference current and points it to the head of the linked list. Line 10 begins a while loop that continues as long as current refers to a valid node, so if the linked list is empty, current is already None and the loop is skipped.

During each iteration, line 11 increases count by one to record the current node. Line 12 then moves current to current.next, allowing the traversal to continue through the list. When current finally becomes None, the loop ends, and line 14 returns count, which is the total number of nodes in the linked list.

Adding a Size Attribute to the LinkedList Class

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted

We've now seen that repeatedly traversing a linked list just to find its size is not the most efficient approach. A better solution is to store the current number of nodes as part of the LinkedList class itself, so the size is always readily available.

Instead of counting the nodes every time, we maintain a dedicated size attribute. Whenever a node is inserted, we increase this value by one, and whenever a node is removed, we decrease it by one, ensuring it always matches the actual number of nodes in the linked list.

With this small improvement, finding the size becomes much simpler because there is no need to traverse the linked list each time. The only responsibility is to keep the size attribute updated after every insertion and deletion, so it always reflects the actual number of nodes stored in the linked list.

The Linked List



- Just introduce a new member in the linked list class, *size*
- Initialize *size* as zero
- When you add or remove a node, increase or decrease ***size*** by one accordingly

```
1 class ListNode:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9         self.size = 0
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted



Now that we have identified the limitation of counting nodes repeatedly, we can improve our linked list with one small change. We simply introduce a new member variable called size, which keeps track of the current number of nodes in the linked list.

When a new linked list is created, the list is empty, so size is initialised to zero. From that point onwards, every successful insertion increases size by one, while every successful deletion decreases it by one, ensuring the stored value always remains accurate.

Because the linked list maintains this value automatically, there is no need to count the nodes whenever the size is requested. The getSize method can simply return the value stored in self.size, making the implementation simpler while avoiding repeated traversal of the linked list.

Size of The LinkedList (Better Solution)

```
1  class ListNode:
2      def __init__(self, data):
3          self.data = data
4          self.next = None
5
6  class LinkedList:
7      def __init__(self):
8          self.head = None
9          self.size = 0
10
11 def sizeList(ll): # ll = LinkedList() #
12   return ll.size
13
14 ll = LinkedList() # Example usage #
11 print(sizeList(ll))
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.
Classification: Restricted



Our improved design makes finding the size of a linked list much simpler than before. Instead of counting the nodes every time, the linked list already keeps track of its current size using the size attribute.

When a new linked list is created, self.size is initialised to zero because the list is empty. After that, every successful insertion increases self.size by one, and every successful deletion decreases it by one, so the stored value always matches the actual number of nodes.

The sizeList function now becomes very straightforward because it simply returns ll.size. There is no need to traverse the linked list or count the nodes again, since the value has already been maintained throughout every insertion and deletion. This makes the implementation both simpler and more efficient than the previous approach.

findNode: Without and With a Size Attribute

```
1  def findNode1(self, index):
2      if self.head is None or index < 0:
3          return None
4
5      cur = self.head
6      while index > 0:
7          cur = cur.next
8          if cur is None:
9              return None
10         index -= 1
11     return cur
```

```
1  def findNode2(self, index):
2      # Validate the index before traversal
3      if self.head is None or index < 0 or index >= self.size:
4          return None
5
6      # Start traversing from head
7      cur = self.head
8      while index > 0:
9          cur = cur.next
10         index -= 1
11     return cur
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

The two versions of findNode perform exactly the same task, but they validate the index differently. findNode1 does not know the size of the linked list, whereas findNode2 uses the size attribute that we introduced earlier.

In findNode1, the method only checks whether the list is empty or the index is negative before starting the traversal. Because it cannot determine whether the index is too large in advance, it must repeatedly check whether cur becomes None while moving through the linked list.

The improved findNode2 performs one additional validation before the traversal begins by checking whether index is greater than or equal to self.size. Once the index is known to be valid, the method can move directly to the requested position without repeatedly checking for None, making the implementation shorter, cleaner, and easier to follow.

InsertNode : Without and With a Size Attribute

```
1  def insertNode1(self, index, data):
2      new_node = ListNode(data)
3
4      if index == 0:
5          new_node.next = self.head
6          self.head = new_node
7          return True
8
9      pre = self.findNode1(index - 1)
10
11     if pre is not None:
12         new_node.next = pre.next
13         pre.next = new_node
14         return True
15
16     return False
```

```
1  def insertNode2(self, index, data):
2      new_node = ListNode(data)
3
4      if index == 0:
5          new_node.next = self.head
6          self.head = new_node
7          self.size += 1
8          return True
9
10     pre = self.findNode2(index - 1)
11
12     if pre is not None:
13         new_node.next = pre.next
14         pre.next = new_node
15         self.size += 1
16         return True
17
18     return False
```

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: F d

Let's compare these two implementations to see what has changed and what has stayed exactly the same. When the index is zero, the new node first points to the current head, and the head is then updated to point to the new node.

For every other position, the process is also identical. Both methods locate the predecessor using **findNode1()** or **findNode2()**, connect the new node to the successor, and then update the predecessor to point to the new node. If no valid predecessor is found, the insertion fails and the method returns False.

The only differences appear in the improved implementation. It uses findNode2, which validates the index using the stored size attribute before traversal begins. After every successful insertion, self.size is increased by one, ensuring that the stored size always remains accurate.

RemoveNode: Without and With a Size Attribute

```
1 def removeNode1(self, index):
2
3     if self.head is None or index < 0:
4         return False
5
6     if index == 0:
7         self.head = self.head.next
8         return True
9
10    pre = self.findNode1(index - 1)
11
12    if pre is None or pre.next is None:
13        return False
14
15    pre.next = pre.next.next
16
17    return True
```

```
1 def removeNode2(self, index):
2
3     if self.head is None or index < 0 or index >= self.size:
4         return False
5
6     if index == 0:
7         self.head = self.head.next
8
9     else:
10        pre = self.findNode2(index - 1)
11        pre.next = pre.next.next
12
13    self.size -= 1
14
15    return True
```



© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

Although these two implementations look very similar, there are a few important differences. When the index is zero, the head is updated to point to the second node. For every other position, the method locates the predecessor and updates its next reference to bypass the node being removed.

The first implementation performs more validation during execution. It checks whether the list is empty or the index is negative, then calls findNode1() and performs additional checks to confirm that both the predecessor and the target node exist. If either check fails, the method returns False.

The improved implementation makes two important changes. It uses findNode2, which validates the index using the stored size attribute before traversal begins. After every successful removal, self.size is decreased by one, ensuring that the stored list size always remains accurate without requiring a separate traversal.

Real world Application – Task Scheduling

- **Task Scheduling :**
  - Linked lists are often used in real-world task scheduling, especially in operating systems and other environments where efficient management of tasks or processes is critical.

- **Advantages of Linked Lists in Task Scheduling:**
  - Efficient Insertion/Deletion: Linked lists allow for quick insertion and deletion of tasks, which is essential in environments where tasks frequently enter and exit the queue.
  - Dynamic Memory Allocation: No pre-allocation of memory is needed, making linked lists ideal for systems with fluctuating task loads

© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

One common real-world use of linked lists is task scheduling in operating systems. Each task can be represented as a node, while the links determine the order in which tasks are processed or executed.

The linked list operations we have just learned are applied directly in this situation. New tasks can be inserted as they arrive, completed tasks can be removed, and the scheduler can traverse the list to find the next task to run. These updates only require changing a few references, making task management both simple and efficient.

Another important advantage is the linked list's ability to grow and shrink dynamically. Unlike an array, memory does not need to be reserved for a fixed number of tasks in advance. This flexibility makes linked lists well suited for systems where the workload changes continuously throughout execution.

Summary



- Linked lists store data in nodes connected through references.
- Unlike arrays, linked-list nodes can occupy non-contiguous memory locations.
- A ListNode stores an item and a reference to the next node.
- A LinkedList manages the head reference and maintains the list size.
- Core operations include:
  - Displaying and traversing the list
  - Finding a node by index
  - Inserting a node at the front, middle, or back
  - Removing a node from a specified position
- The size attribute is updated after every successful insertion or removal.

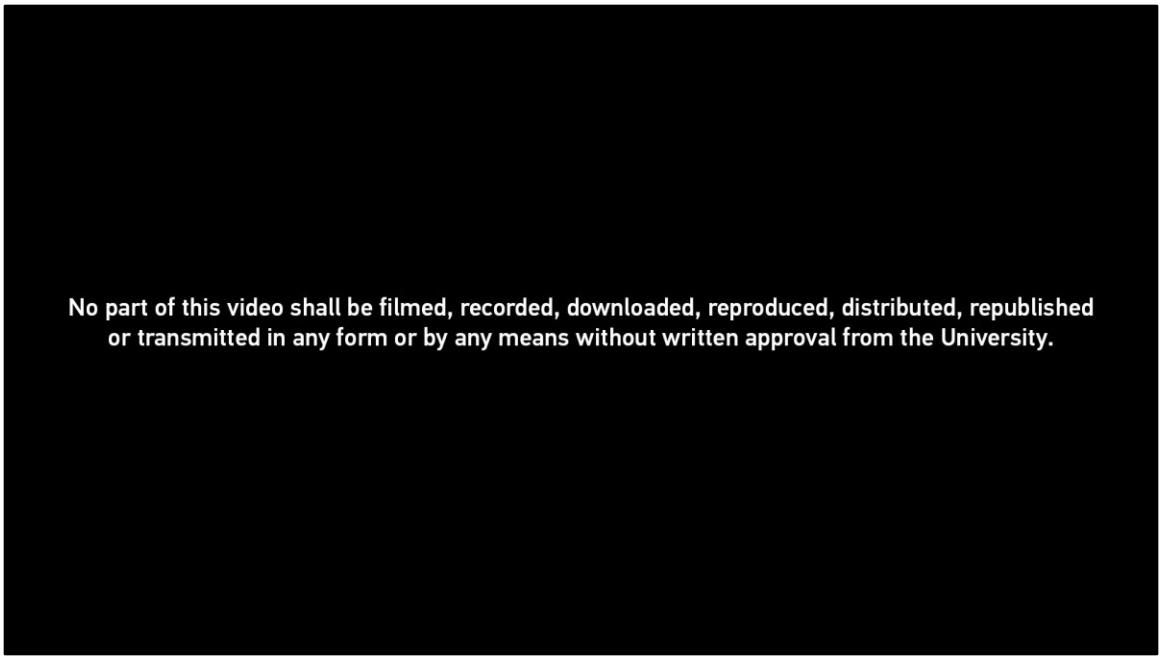


© 2025 Nanyang Technological University, Singapore. All Rights Reserved. Classification: Restricted

This brings us to the end of our introduction to singly linked lists. We have seen how a linked list stores data in separate nodes connected through references, allowing those nodes to be stored in non-contiguous memory locations.

Along the way, we developed two main classes. The ListNode class stores an item and a reference to the next node, while the LinkedList class manages the head reference and maintains the size of the list.

We also implemented the fundamental linked-list operations, including traversal, searching, insertion, and deletion. By maintaining the size attribute after every successful insertion and removal, we can retrieve the list size without counting the nodes each time. In the next topic, we will extend these ideas by exploring more advanced linked-list structures and their practical applications.


No part of this video shall be filmed, recorded, downloaded, reproduced, distributed, republished
or transmitted in any form or by any means without written approval from the University.